Creación de una aplicación FIWARE

Práctica 1 – Gestión de Datos en Entornos Inteligentes

Objetivo

Construir una primera aplicación sencilla FIWARE usando Google Antigravity / GitHub Copilot.

Entrega

Un archivo ZIP con la carpeta de la aplicación web obtenido **mediante *git archive***.

Ajusta *.gitignore* para que no se incluyan ni la carpeta *.venv* ni las carpetas __pycache__ en el archivo ZIP. En cambio, **incluye un fichero *requirements.txt*** obtenido ejecutando *pip freeze > requirements.txt*.

Tecnologías a usar

HTML y CSS

- Tutorial Mozilla Developer Network (MDN) Getting started with the web
- W³ Schools HTML Tutorial, W³ Schools CSS Tutorial

Framework Web Flask

Utilizaremos Flask, uno de los dos principales *frameworks* de desarrollo web en Python (el otro es Django).

Desarrollo de una aplicación de gestión de tiendas y productos

Crea una nueva aplicación Flask denominada *fiware-smart-store* siguiendo esta secuencia de tickets. Recuerda decirle siempre al final de cada ticket que actualice *PRD.md, architecture.md* y *data_model.md*.

Ticket #1 – Crear una aplicación de gestión de una cadena de supermercados.

- Revisar *TASK.md* e *implementation_plan.md*, hacer comentarios, pedir cambios.
- Cuando el agente finalice, pedirle que cree los archivos *PRD.md*, *architecture.md* y *data_model.md.*

Ticket #2 – Añadir soporte multi-idioma

Ticket #3 – Añadir batería de tests

Ticket #4 – Cambiar el modelo de datos

- El nuevo modelo de datos es el del tutorial FIWARE NGSIv2 *CRUD Operations*.
- Añadir un atributo *image* tanto a *Product* como a *Store* y un atributo *originCountry* a *Product* para indicar el país de origen de un producto.
- Asegurarse de que la aplicación tiene 3 vistas principales: *dashboard*, tiendas y productos. Adicionalmente, habrá una vista de detalle de una tienda y otra de detalle de un producto. También puede haber formularios adicionales para añadir / modificar entidades.

Ticket #5 – Añadir tipo de entidad *Employee (id, name, image, salary, role, refStore)*

Ticket #6 – Mejoras UI

- Incluir en la vista de detalle de una tienda un mapa con su ubicación.
- Asegurarse de que todas las imágenes de tiendas, productos y empleados se muestran correctamente.
- Asegurarse de que todos los botones y enlaces de la app funcionan.

Ticket #7 – Operaciones CRUD

- Asegurarse de que en la vista de detalle de una tienda se muestre la lista de productos almacenados en la tienda y se permita añadir / modificar /borrar un ítem de inventario. También debe permitirse añadir / modificar / borrar una balda.
- Asegurarse de que en la vista de detalle de un producto se muestre la lista de tiendas donde está disponible y se permita añadir / modificar / borrar un ítem de inventario.

Ticket #8 – Integración con Orion

- Añadir integración con Orion como alternativa a la BD SQLite usada hasta ahora.
- NO hay que sincronizar la BD SQLite con Orion. Simplemente la aplicación debe comprobar la conectividad con Orion al arranque. Si no hay conectividad con Orion usará la BD SQLite.
- Copiar el fichero *docker-compose.yml* del tutorial *CRUD Operations* a la carpeta de la aplicación. Crear un *script* que permita levantar la aplicación y los contenedores y otro que permita tirarlos abajo. El *script* que levanta la aplicación debe tirar abajo los contenedores que se estén ejecutando antes de levantarlos.

Criterios de evaluación

- Interacción adecuada con el agente y correctamente estructurada a lo largo de los tickets.
- Funcionalidades cubiertas.
- Apariencia estética de la aplicación.